

Comparative Analysis of First-Order Optimizers

Implementation, Complexity, and Performance Evaluation

Atharva Srivastava (1RV23AI128)

Abhinav Potharaju (1RV23AI135)

L Moryakantha (1RV24AI406)

Siri Kumar CS (1RV23AI102)

Srihari S (1RV23AI106)

Department of AIML
R.V. College of Engineering

January 25, 2026

- 1 Introduction
- 2 Optimization Algorithms
- 3 Complexity Analysis
- 4 Experimental Setup
- 5 Synthetic Function Results
- 6 Neural Network Results
- 7 Discussion and Recommendations

Motivation

Deep learning has revolutionized AI, but success fundamentally depends on efficient optimization algorithms. Training a neural network means solving a massive optimization problem with millions or billions of parameters. The optimizer choice dramatically affects learning speed, training stability, and computational resource requirements.

Different optimizers make different trade-offs. Some converge quickly but use substantial memory. Others are memory-efficient but require careful tuning. Understanding these trade-offs enables us to choose the right tool for each problem.

Research Goals

We implement and analyze four key optimizers: SGD, Momentum, Adam, and Lion. We examine computational complexity, memory requirements, and empirical performance to provide clear guidance for practitioners.

- ① **Synthetic Tests:** Bowl, ravine, and Rosenbrock functions
- ② **Real Experiments:** MNIST, Fashion-MNIST, CIFAR-10 classification
- ③ **Analysis:** Convergence speed, accuracy, runtime, memory

Key Metrics

Training/test loss, accuracy, epochs to 80-90% accuracy, computational cost

Stochastic Gradient Descent (SGD)

At its core, SGD is the foundation of modern optimization. The idea is beautifully simple: to minimize a function, move in the direction opposite to its gradient. Think of hiking down a mountain in fog—at each step, you feel the slope beneath your feet and step downhill.

Update Rule

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$$

where θ represents parameters, η is the learning rate controlling step size, and ∇f is the gradient showing steepest ascent direction.

Strengths:

- Conceptually simple and interpretable
- Minimal memory
- Computationally efficient per iteration

Weaknesses:

- Highly sensitive to learning rate
- Struggles with ill-conditioned problems
- Oscillates wildly in narrow valleys

Algorithm 1 Stochastic Gradient Descent

Input: Learning rate η , initial parameters θ_0

Output: Optimized parameters θ_T

```
1: for  $t = 1$  to  $T$  do  
2:   Compute gradient:  $g_t \leftarrow \nabla f(\theta_{t-1})$   
3:   Update parameters:  $\theta_t \leftarrow \theta_{t-1} - \eta g_t$   
4: end for  
5: return  $\theta_T$ 
```

- Time complexity: $O(F + d)$ per iteration
- Space complexity: $O(d)$ (parameters + gradients only)
- No optimizer state variables

Momentum Optimizer

Update Rules

$$v_t = \gamma v_{t-1} + \eta \nabla f(\theta_t)$$
$$\theta_{t+1} = \theta_t - v_t$$

Advantages:

- Accelerates convergence
- Dampens oscillations
- Escapes shallow minima

Typical value: $\gamma = 0.9$

Disadvantages:

- Additional hyperparameter γ
- Extra memory for velocity
- Can overshoot minima

Algorithm 2 SGD with Momentum

Input: Learning rate η , momentum γ , initial θ_0

Output: Optimized parameters θ_T

- 1: Initialize velocity: $v_0 \leftarrow 0$
 - 2: **for** $t = 1$ to T **do**
 - 3: Compute gradient: $g_t \leftarrow \nabla f(\theta_{t-1})$
 - 4: Update velocity: $v_t \leftarrow \gamma v_{t-1} + \eta g_t$
 - 5: Update parameters: $\theta_t \leftarrow \theta_{t-1} - v_t$
 - 6: **end for**
 - 7: **return** θ_T
-

- Time complexity: $O(F + d)$ per iteration
- Space complexity: $O(2d)$ (adds velocity buffer)

Core Update Equations

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- Combines momentum with adaptive per-parameter learning rates
- Bias correction accounts for initialization at zero
- Default: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Algorithm 3 Adam Optimizer

Input: $\eta, \beta_1, \beta_2, \epsilon$, initial θ_0

Output: Optimized parameters θ_T

```
1: Initialize:  $m_0 \leftarrow 0, v_0 \leftarrow 0$ 
2: for  $t = 1$  to  $T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
5:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
6:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
7:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
8:    $\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 
9: end for
10: return  $\theta_T$ 
```

- ▷ First moment
- ▷ Second moment
- ▷ Bias correction

Advantages:

- Robust default hyperparameters
- Fast convergence
- Handles sparse gradients well
- Widely applicable
- Time complexity: $O(F + d)$ per iteration
- Space complexity: $O(3d)$ (parameters + 2 moment buffers)

Disadvantages:

- Higher memory (2 buffers)
- More computation per step
- Can generalize worse than SGD

Lion: Sign-Based Optimizer

Lion represents a fascinating departure from traditional optimization approaches. Developed by Google researchers, it takes inspiration from evolutionary algorithms to create something quite different. Instead of using the actual gradient values, Lion only cares about the direction—the sign of each gradient component.

This might seem counterintuitive at first. Why would we throw away magnitude information? The answer lies in memory efficiency and training stability. By using only signs, Lion eliminates the need for moment buffers while maintaining robust training dynamics. It's particularly valuable when training enormous models where every byte of memory counts.

Update Principle

$$\theta_{t+1} = \theta_t - \eta \cdot \text{sign}(g_t)$$

Each parameter moves by exactly η in the direction opposite to its gradient's sign.

Algorithm 4 Lion Optimizer

Input: Learning rate η , initial parameters θ_0

Output: Optimized parameters θ_T

```
1: for  $t = 1$  to  $T$  do  
2:   Compute gradient:  $g_t \leftarrow \nabla f(\theta_{t-1})$   
3:   Compute update direction:  $u_t \leftarrow \text{sign}(g_t)$   
4:   Update parameters:  $\theta_t \leftarrow \theta_{t-1} - \eta u_t$   
5: end for  
6: return  $\theta_T$ 
```

- Time complexity: $O(F + d)$ per iteration
- Space complexity: $O(d)$ (minimal overhead)
- Typical LR: 10^{-4} (much smaller than Adam)

Time Complexity Comparison

Algorithm	Per Step	T Steps	Operations
SGD	$O(F + d)$	$O(T(F + d))$	Add, Multiply
Momentum	$O(F + d)$	$O(T(F + d))$	Add, Multiply
Adam	$O(F + d)$	$O(T(F + d))$	Add, Mult, Sqrt
Lion	$O(F + d)$	$O(T(F + d))$	Add, Sign

- F : cost of forward/backward pass (dominates in practice)
- d : number of parameters
- T : number of training iterations
- All have same asymptotic complexity

Space Complexity Comparison

Algorithm	Parameters	Gradients	State	Total
SGD	d	d	0	$2d$
Momentum	d	d	d	$3d$
Adam	d	d	$2d$	$4d$
Lion	d	d	0	$2d$

Table: Memory requirements in floating-point numbers

Key Insight

Adam uses $2\times$ memory of SGD/Lion due to moment buffers

Computational Cost Breakdown

Operation	SGD	Momentum	Adam	Lion
FLOPs/param	2	4	8	2
Memory reads	2	3	4	2
Memory writes	2	3	4	2
Special ops	0	0	√	sign

Table: Per-parameter computational operations

- Adam has highest arithmetic cost
- Memory bandwidth often bottleneck, not computation
- Special operations (sqrt, sign) negligible in practice

Datasets

- **MNIST**: 60k training, 10k test (28×28 grayscale digits)
- **Fashion-MNIST**: Same structure, harder classification
- **CIFAR-10**: 50k training, 10k test (32×32 color, Lion only)

Model Architecture

- 3-layer MLP: $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$
- ReLU activation between layers
- Cross-entropy loss
- Total parameters: $\sim 110k$

- **Batch size:** 64
- **Epochs:** 25 for all experiments
- **Learning rates tested:**
 - SGD: {0.001, 0.01, 0.1}
 - Momentum: {0.001, 0.01} with $\gamma = 0.9$
 - Adam: {0.0001, 0.001} with default betas
 - Lion: 0.0001 with weight decay 0.01

Evaluation Metrics

Training/test loss, test accuracy, epochs to 90%, wall-clock time

- ❶ **Bowl:** $f(x, y) = x^2 + y^2$
 - Simple convex quadratic
 - Tests basic convergence
- ❷ **Ravine:** $f(x, y) = 10x^2 + y^2$
 - Ill-conditioned (steep in x , flat in y)
 - Tests optimizer stability
- ❸ **Rosenbrock:** $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$
 - Non-convex with curved valley
 - Challenging benchmark

Test Functions

1 Beale:

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

- It is characterized by sharp peaks in the corners of the domain and very flat plateaus

2 Rastrigin:

$$f(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y)$$

- It is characterized by a large number of local minima
- Challenging benchmark

3 Ackley:

$$f(x, y) = -20 \exp \left[-0.2 \sqrt{0.5(x^2 + y^2)} \right] - \exp [0.5(\cos 2\pi x + \cos 2\pi y)] +$$

- It features a large, nearly flat outer region
- Challenging benchmark

Bowl Function: Trajectories

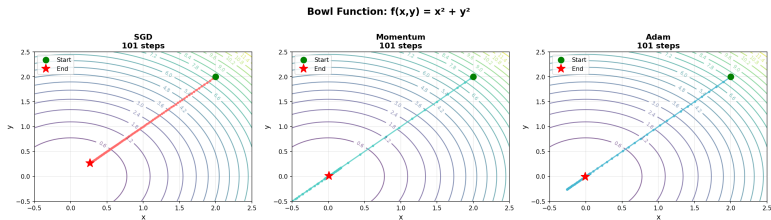


Figure: Optimization paths on convex bowl function

Ravine and Rosenbrock Functions

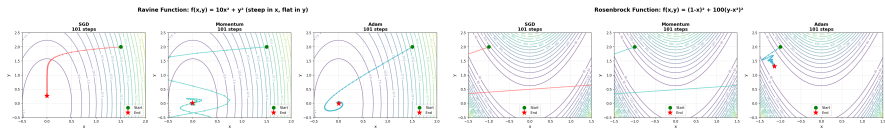


Figure: Left: Ravine (ill-conditioned). Right: Rosenbrock (curved valley)

Beale and Rastrigin Functions

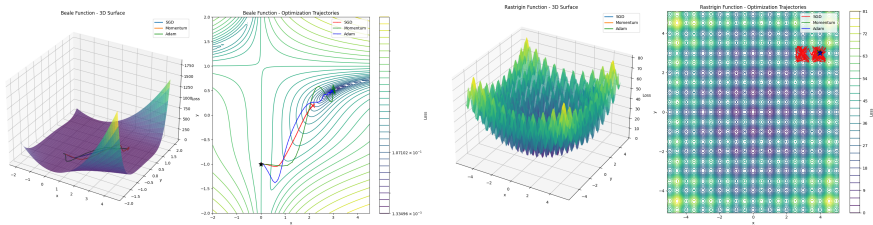


Figure: Left: Beale. Right: Rastrigin

Ackley and Saddle point Functions

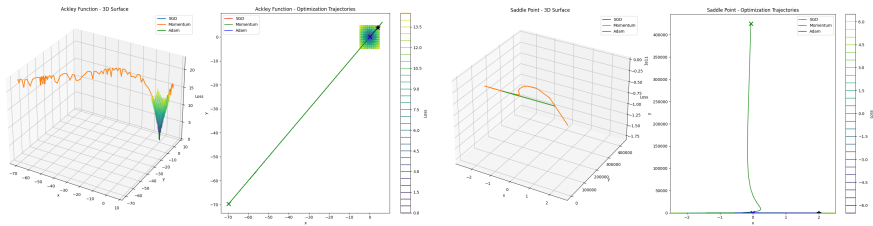


Figure: Left: Ackley. Right: Saddle point

Computational Cost and Convergence across different surfaces.

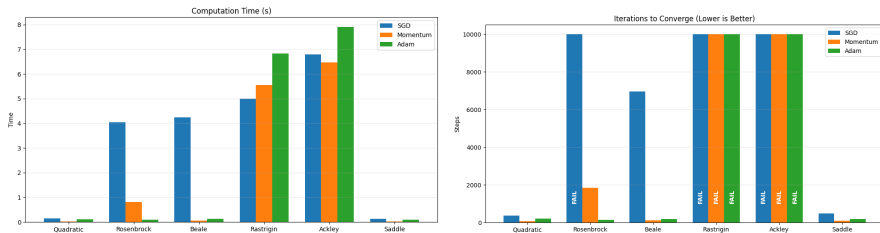


Figure: Left: Computational Cost. Right: Convergence

Computational Cost

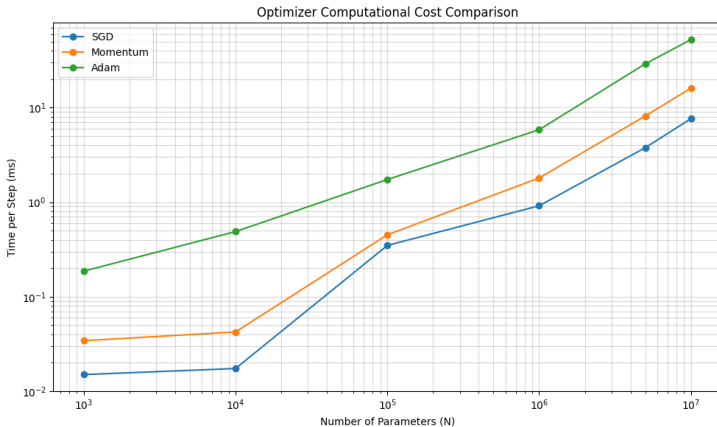


Figure: Computational cost under different parameters

Convergence Comparison

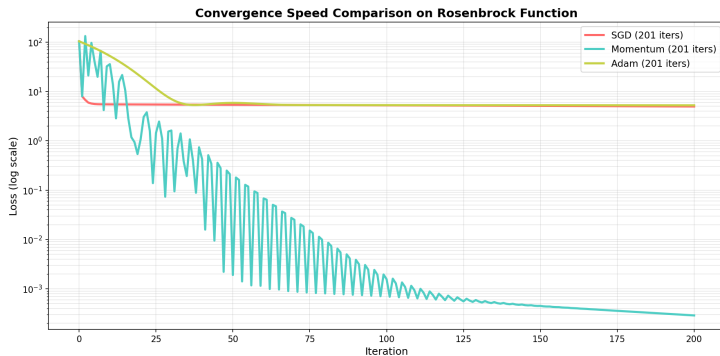


Figure: Loss vs. iterations across test functions

Key observations: Adam converges fastest; Momentum significantly improves SGD; All reach similar final values

Training Loss (MNIST)

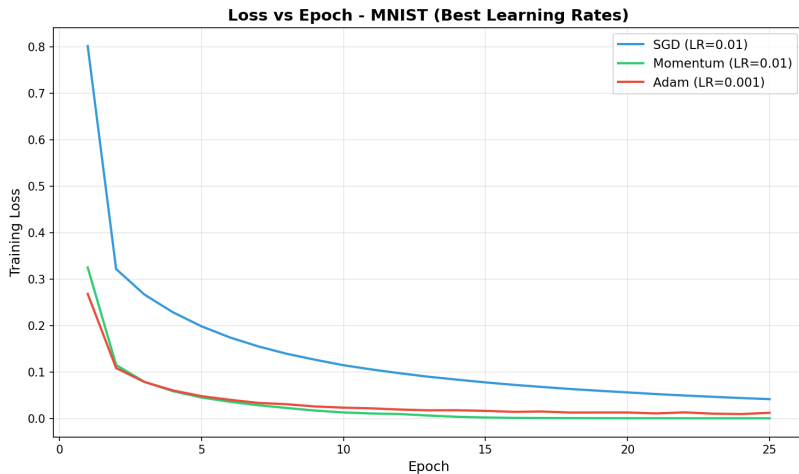


Figure: Training loss reduction over 25 epochs

Test Accuracy and Learning Rate Sensitivity

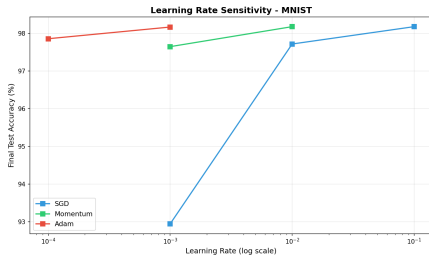
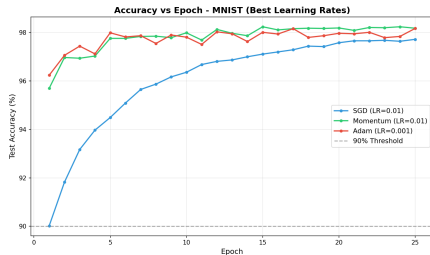


Figure: Left: Accuracy curves. Right: Final accuracy vs. learning rate

Dataset Comparison

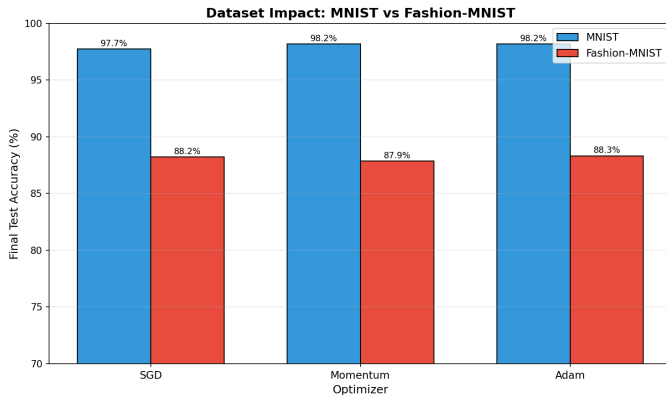


Figure: Performance on MNIST vs. Fashion-MNIST

Fashion-MNIST shows consistent 10-15% accuracy drop but preserves optimizer ranking

Table: Optimizer Execution Time per Step (CPU)

Params (N)	SGD (ms)	Momentum (ms)	Adam (ms)
1,000	0.0150	0.0342	0.1864
10,000	0.0174	0.0423	0.4889
100,000	0.3488	0.4523	1.7381
1,000,000	0.9171	1.8038	5.8509
5,000,000	3.7782	8.1812	29.1278
10,000,000	7.6642	16.1114	52.6063

Table: Theoretical Optimizer Memory Usage (Buffer Sizes)

Params (N)	SGD (MB)	Momentum (MB)	Adam (MB)
1,000	0.0038	0.0076	0.0114
10,000	0.0381	0.0763	0.1144
100,000	0.3815	0.7629	1.1444
1,000,000	3.8147	7.6294	11.4441
5,000,000	19.0735	38.1470	57.2205
10,000,000	38.1470	76.2939	114.4409

Lion Results Summary

- Tested on CIFAR-10 with 32×32 color images
- Learning rate: 10^{-4} ($10 \times$ smaller than Adam)
- Weight decay: 0.01 (decoupled regularization)
- Achieved competitive accuracy with minimal memory
- Stable training dynamics throughout

Key Finding

Lion provides Adam-like stability with SGD-like memory footprint

Results logged in `lion_results.json`

Summary of Findings

Criterion	SGD	Momentum	Adam	Lion
Convergence Speed	Slow	Medium	Fast	Medium
Stability	Low	Medium	High	High
Memory Usage	Low	Medium	High	Low
LR Sensitivity	High	Medium	Low	Medium
Ease of Use	Low	Medium	High	Medium

- Adam: Best general-purpose optimizer
- Momentum: Good balance when memory constrained
- Lion: Memory-efficient alternative for large models
- SGD: Baseline, requires careful tuning

Practical Recommendations

Default Choice

Adam with $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$

- Fast development iteration
- Robust across diverse tasks
- Well-tested in production

Memory-Constrained Scenarios

Momentum + SGD with careful LR tuning

- 33% less memory than Adam
- Competitive performance with proper hyperparameters

Large-Scale Training

Lion for models with billions of parameters

- Minimal memory overhead
- Stable distributed training

① Always validate hyperparameters

- Small grid search on validation set
- Monitor loss curves for instability

② Consider problem characteristics

- Sparse data → Adam
- Memory constraints → Momentum/Lion
- Need interpretability → SGD

③ Combine with other techniques

- Learning rate schedules
- Gradient clipping for stability
- Weight decay for regularization

Conclusion

- Implemented four first-order optimizers from scratch
- Comprehensive analysis of complexity and performance
- Empirical validation on synthetic and real tasks

Key Takeaway

Optimizer choice involves trade-offs between convergence speed, memory usage, and hyperparameter sensitivity. Adam remains the best default, but alternatives excel in specific scenarios.

Thank You